



中国科学技术大学
University of Science and Technology of China



分布式算法

2025年5月23日

大数据算法：分布式算法

- 基础概念
- 算法的分布式实现
- 分布式机器学习训练
- 联邦学习与区块链



大数据算法：分布式算法

- 基础概念
- 算法的分布式实现
- 分布式机器学习训练
- 联邦学习与区块链



基础概念——为什么需要分布式算法？

1. 单机计算能力增长的“天花板”

摩尔定律放缓

- 过去几十年里，摩尔定律预测芯片上晶体管数量每18–24个月翻倍，带来性能飞跃。但近年来，这一趋势**明显放缓**，因为制程工艺接近物理极限（比如3nm以下制程面临量子隧穿问题）。

主频提升受限

- 主频（GHz）在2010年前后基本达到了上限（ $\approx 4\text{GHz}$ ），继续提升会带来更高功耗，以及**巨大的发热量**（散热成本飙升）。单核心性能提升变得越来越困难，转而依赖“**多核并行**”，但多核也受限于缓存一致性与带宽瓶颈。

基础概念——为什么需要分布式算法？

2. 存储和I/O的瓶颈

内存带宽瓶颈

- 即便CPU计算能力强，如果**无法及时获取数据**（I/O瓶颈），依然无法高效运行。
- **内存墙**（memory wall）现象：CPU性能增长远快于内存带宽增长 → 数据等不来！

存储容量限制

- 单台机器的磁盘、SSD、内存都是有限的。大数据场景下（TB~PB级），根本无法在一台机器上放得下全部数据。

基础概念——为什么需要分布式算法？

3. 单机扩展能力有限

单机扩展 = 垂直扩展 (scale-up)

- 升级更好的CPU/内存/磁盘 → 昂贵且存在极限。

分布式扩展 = 水平扩展 (scale-out)

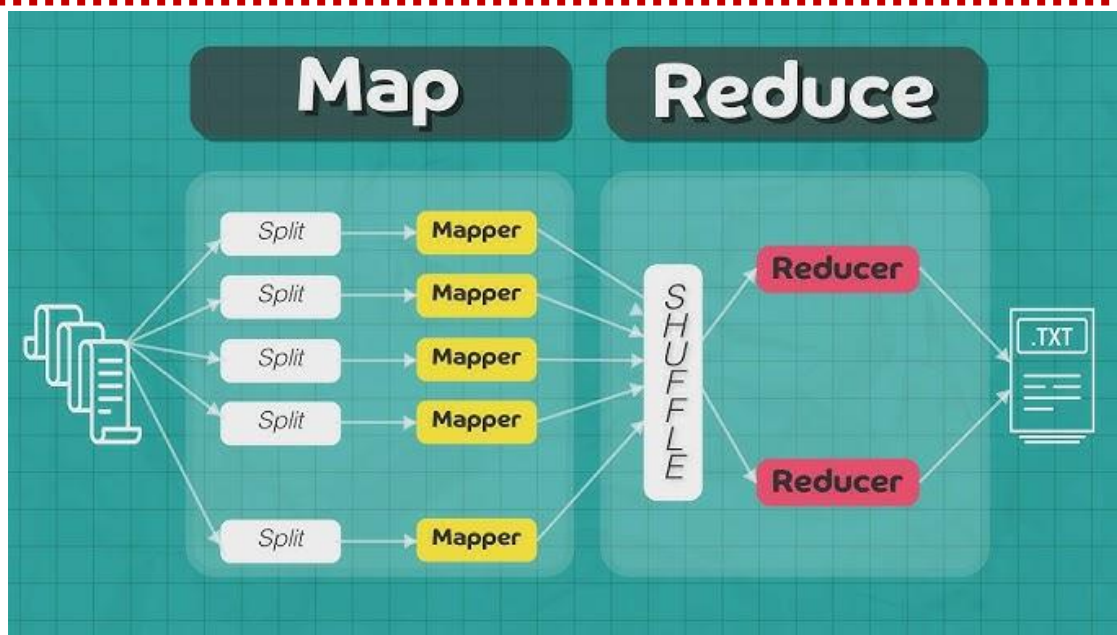
- 多台普通机器协作工作 → 更经济、易扩展。

基础概念：通信复杂度

通信复杂度类型	定义	说明/适用场景
总通信量 (Total Communication Volume)	所有机器之间在整个执行过程中传输的数据总量	常用于评估算法总体开销，MapReduce等模型常见度量方式
每轮通信量 (Per-Round Communication)	每一轮中通信的数据量	适用于多轮算法，分析延迟瓶颈
轮数 (Number of Rounds)	算法执行需要的通信轮数	用于衡量同步/延迟成本
机器负载 (Load per Machine)	每台机器在单轮中所需传输或处理的数据量	衡量最坏负载，防止单点瓶颈
局部计算量 (Local Computation)	每台机器上本地执行的计算量	虽非通信复杂度，但常与通信协同分析

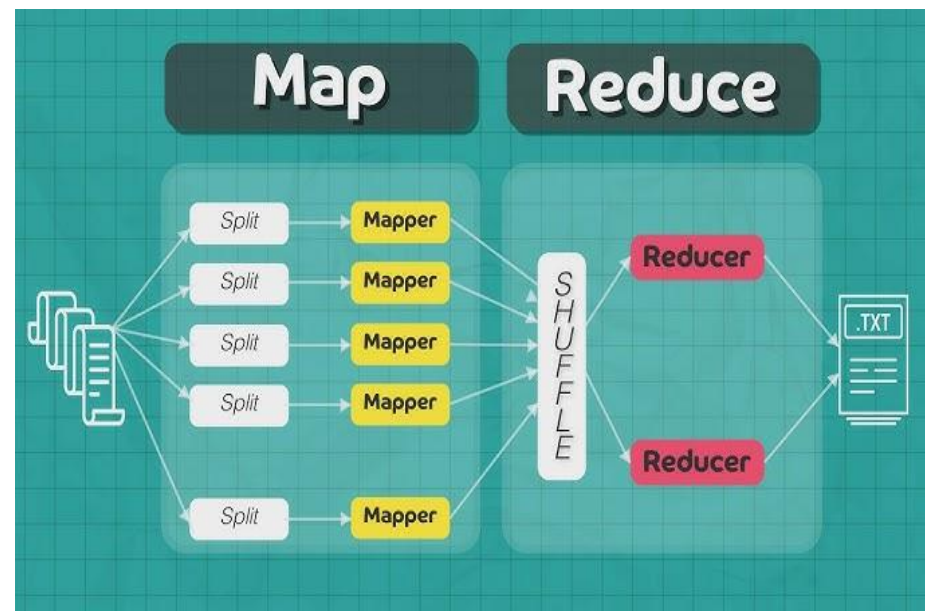
基础概念：MapReduce

- **MapReduce** 是由 Google 提出的一种编程模型和执行框架，用于在分布式环境下高效处理和生成大规模数据集。
- 在处理大规模数据（如 TB 级别甚至 PB 级别）时，**单台机器内存和处理能力受限**。因此需要一种 **能在多台机器上并行执行、自动管理任务分发与容错** 的计算框架。

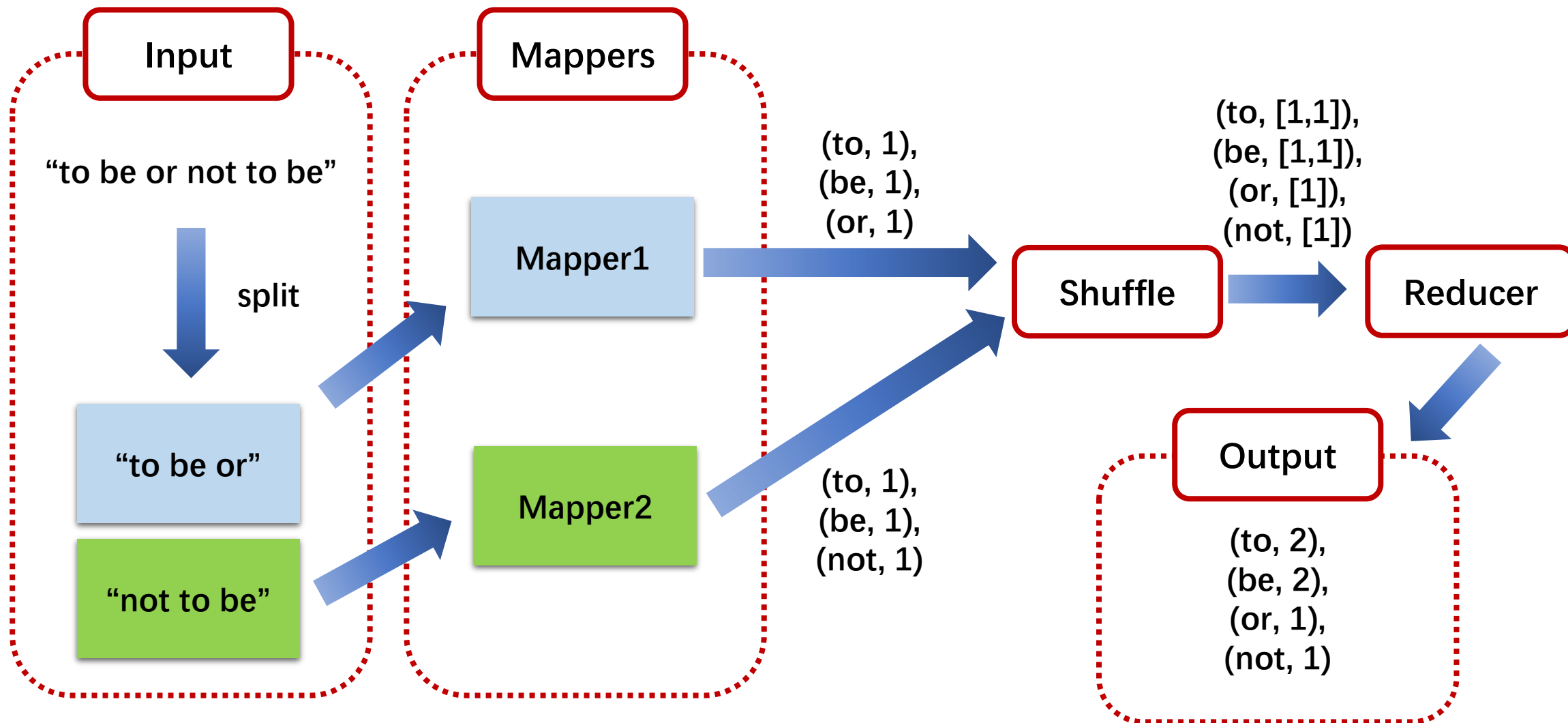


基础概念：MapReduce

- 1.数据划分：**将输入数据分成多个 “split” （块），每块分配给一个 Mapper。
- 2.Map：**各个 Mapper 运行用户定义的 Map() 函数，产生若干的（key，value）对，作为中间结果。
- 3.Shuffle：**系统将相同 key 的中间结果发送到同一个 Reducer。
- 4.Reduce：**Reducer 聚合处理相同 key 的所有值。
- 5.输出结果：**最终结果写入分布式文件系统（如 HDFS）。



MapReduce: 分布式词频统计

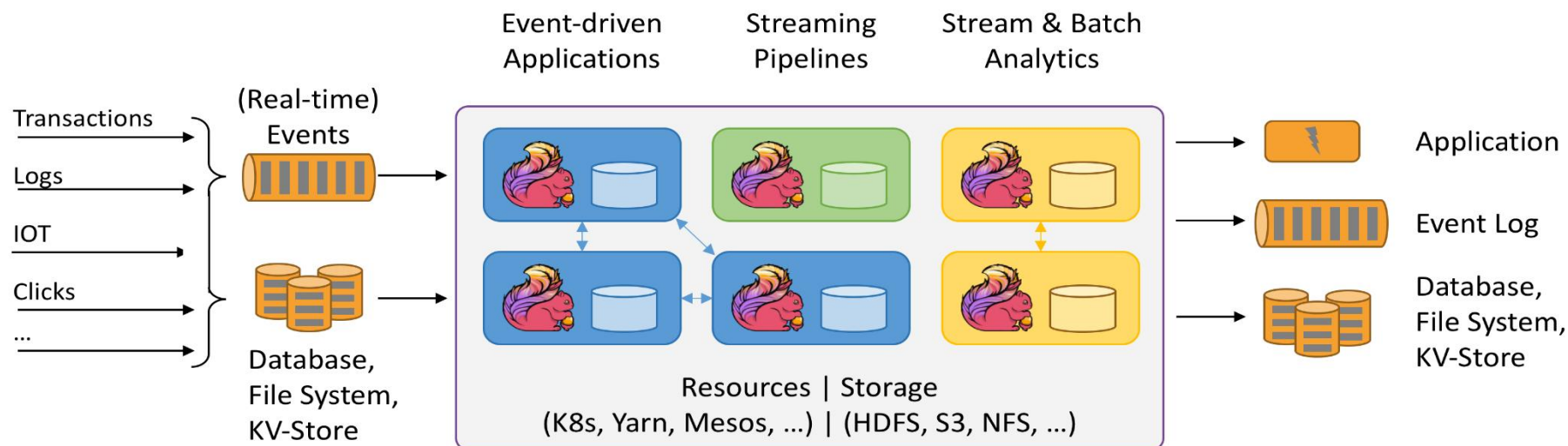


基础概念：MapReduce

虽然 MapReduce 功能强大，但也有一些局限：

- **不适合迭代计算：** 如机器学习训练、图计算；
- **延迟较高：** 不适用于低延迟实时处理；
- **中间结果频繁写磁盘：** 导致 I/O 成本高。

因此，后续出现了 Spark、Flink 等内存计算框架来补足其短板。



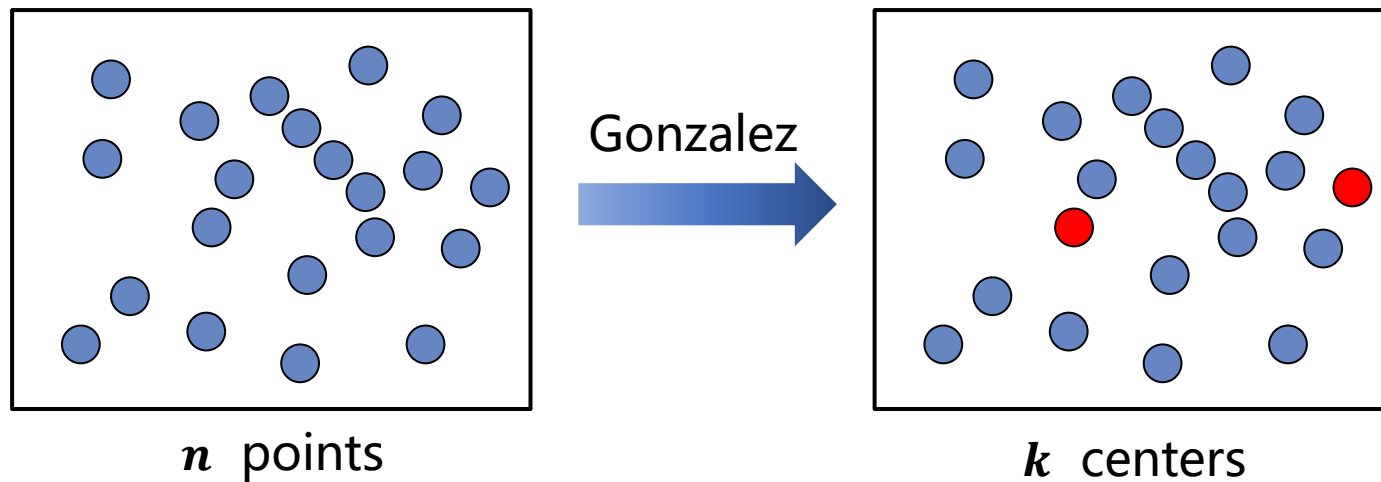
大数据算法：分布式算法

- 基础概念
- **算法的分布式实现**
- 分布式机器学习训练
- 联邦学习与区块链



分布式k-center

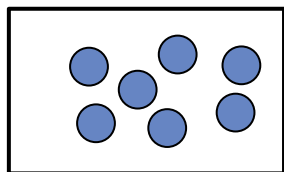
单机版本



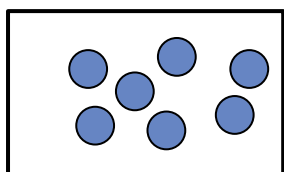
2-approximation,
 $O(nkd)$ processing time

分布式k-center

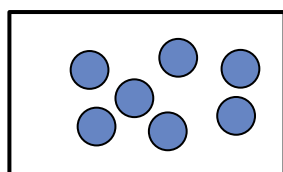
分布式版本



n/m points

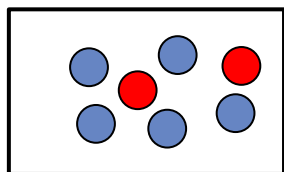


n/m points

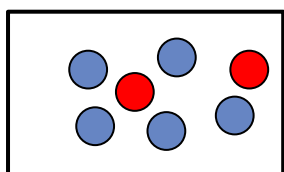


n/m points

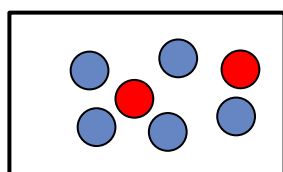
Mappers: Gonzalez



k centers

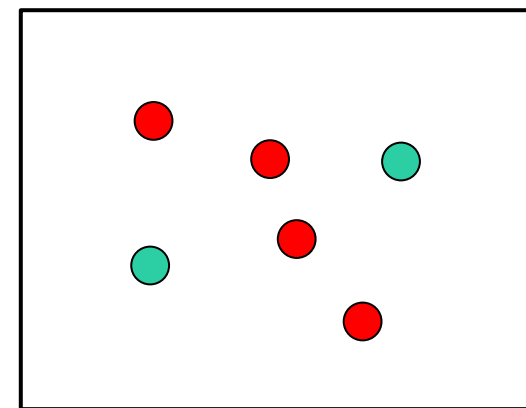


k centers



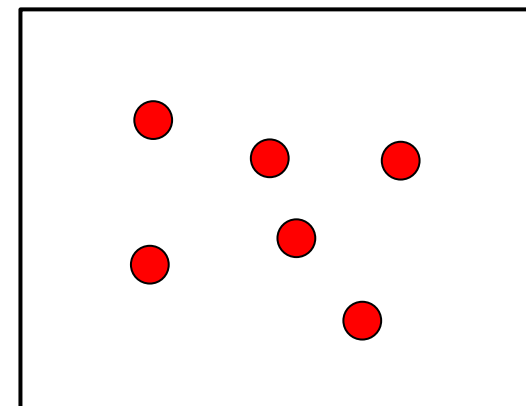
k centers

Shuffle



k centers

Reducer: Gonzalez



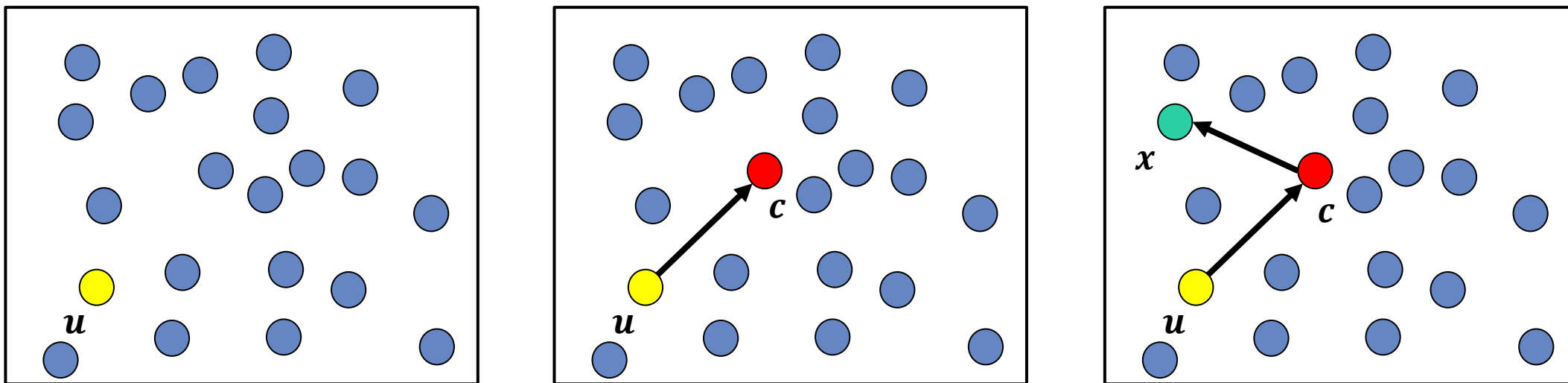
mk centers

4-approximation,

$O(\frac{nk d}{m} + mk^2 d)$ processing time + communication time,

Communicates $O(km)$ amount of data

分布式k-center



反证法可证: $d(u, c) \leq 2OPT$; $d(c, x) \leq 2OPT$

$$d(u, x) \leq d(u, c) + d(c, x) \leq 4OPT$$

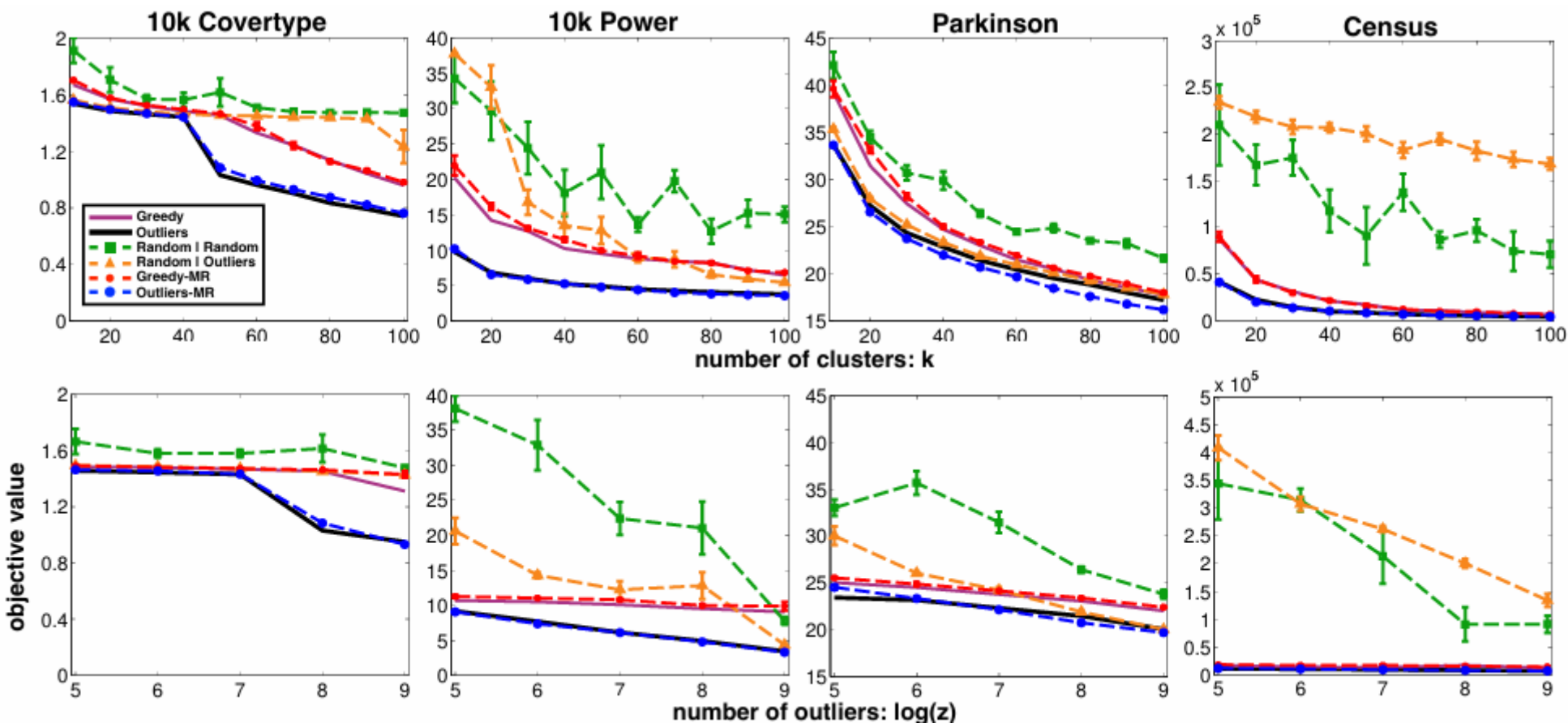
分布式k-center with outliers

- 针对k-center with outliers问题，利用与分布式k-center相同思路设计的算法Outliers-MR[1]

	Sequential Algorithm	Distributed Algorithm (Outliers-MR)
近似比	3	13
总时间复杂度	$O(kn^2)$	$O((k+z)\left(\frac{n}{m}\right)^2 + km^2(k+z)^2)$
通信复杂度	/	$O((k+z)m \cdot \log n)$
MapReduce轮数	/	2
每台机器内存	$O(n)$	$O(\max\{n/m, m(k+z)\log n\})$
可扩展性	差	好

[1] Malkomes G, Kusner M J, Chen W, et al. Fast distributed k-center clustering with outliers on massive data[J]. Advances in Neural Information Processing Systems, 2015, 28.

分布式k-center with outliers



- 针对k-center with outliers问题，Outliers与Outliers-MR[1]的聚类效果几乎相同。

分布式k-center with outliers

	Greedy	Greedy-MR	Outliers	Outliers-MR
近似比	2	4	3	13
总时间复杂度	$O(nkd)$	$O(\frac{nk d}{m} + m k^2 d)$	$O(kn^2)$	$O((k+z) \left(\frac{n}{m}\right)^2 + km^2(k+z)^2)$

- 在理想情况下，分布式k-center与分布式k-center with outliers算法相较于原始算法分别可以实现 m 与 m^2 倍的加速。
- 右图为在4个真实数据集中，分布式算法的加速倍数。
(机器数 $m = 10$)

dataset	k -center	outliers
10k Coverttype	3.6	6.2
10k Power	4.8	9.4
Parkinson	4.9	4.4
Census	12.4	677.7

分布式PCA降维

PCA(主成分分析)

目标:

- 找到数据中方差最大的方向
- 将原始数据投影到这些方向上, 从而实现降维

PCA步骤:

1. 给定中心化后的数据矩阵 $\tilde{X} \in R^{n \times d}$
2. 计算其协方差矩阵 $\Sigma = \frac{1}{n} \tilde{X}^T \tilde{X}$
3. 对协方差矩阵做特征值分解或奇异值分解(SVD)
4. 取前 k 个主成分对应的特征向量/奇异向量

协方差矩阵聚合

1. 第 i 台机器分到的数据为 X_i , 中心化后为 $\tilde{X}_i$
2. 计算**本地**协方差矩阵: $\Sigma_i = \frac{1}{n_i} \tilde{X}_i^T \tilde{X}_i$
3. 各机器上传 Σ_i 并加权合并: $\Sigma = \frac{1}{n} \sum_{i=1}^m \Sigma_i$
4. 对总协方差矩阵做特征分解

子空间合并

1. 第 i 台机器分到的数据为 X_i , 中心化后为 $\tilde{X}_i$
2. 每台机器**本地执行PCA**, 取前 k 个主成分:
$$U_i \in R^{d \times k}$$
3. 各机器上传子空间 U_i 到中心机器
4. 中心机器对这些子空间做合并 (如: 对 $[U_1, \dots, U_m]$ 做SVD), 得到全局主成分

分布式PCA降维

子空间合并

1. 第 i 台机器分到的数据为 X_i ，中心化后为 $\tilde{X}_i$
2. 每台机器**本地执行PCA**，取前 k 个主成分：

$$U_i \in R^{k \times d}$$

3. 各机器**上传子空间** U_i 到中心机器
4. 中心机器**对这些子空间做合并**（如：对 $[U_1, \dots, U_m]$ 做SVD分解），得到全局主成分

- 通信数据量： $O(mkd)$
- 信息丢失严重

改进—减少信息丢失[2]

1. 第 i 台机器分到的数据为 X_i ，中心化后为 $\tilde{X}_i$
2. 每台机器**本地执行保距的数据压缩**（对任意方向的长度近似保持）得到低维数据矩阵：

$$\tilde{X}'_i \in R^{l \times d}$$

3. 各机器**上传低维数据矩阵** $\tilde{X}'_i$ 到中心机器
4. 中心机器**对这些低维数据矩阵做拼接**，再做**Randomized SVD**分解，得到全局主成分

- 通信数据量： $O(mkd/\epsilon^2)$
- 有全局的误差保证： $(1 + \epsilon)$ 倍乘法误差， ϵ 倍加法误差

[2]Liang Y, Balcan M F F, Kanchanapally V, et al. Improved distributed principal component analysis[J]. Advances in neural information processing systems, 2014, 27.

分布式PCA降维

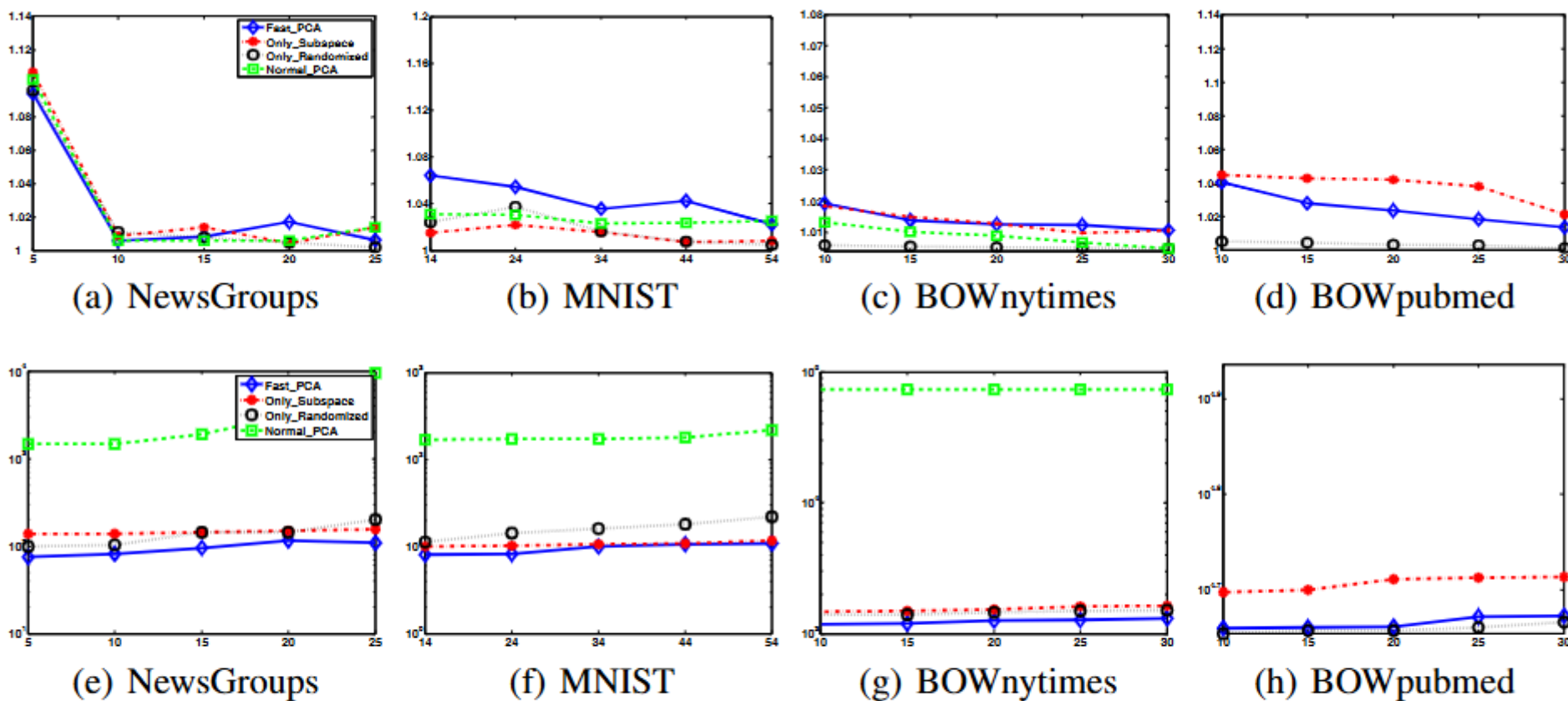


Figure 2: Low rank approximation. First row: error (normalized by baseline) v.s. projection dimension. Second row: time v.s. projection dimension.

[2]Liang Y, Balcan M F F, Kanchanapally V, et al. Improved distributed principal component analysis[J]. Advances in neural information processing systems, 2014, 27.

分布式PCA降维

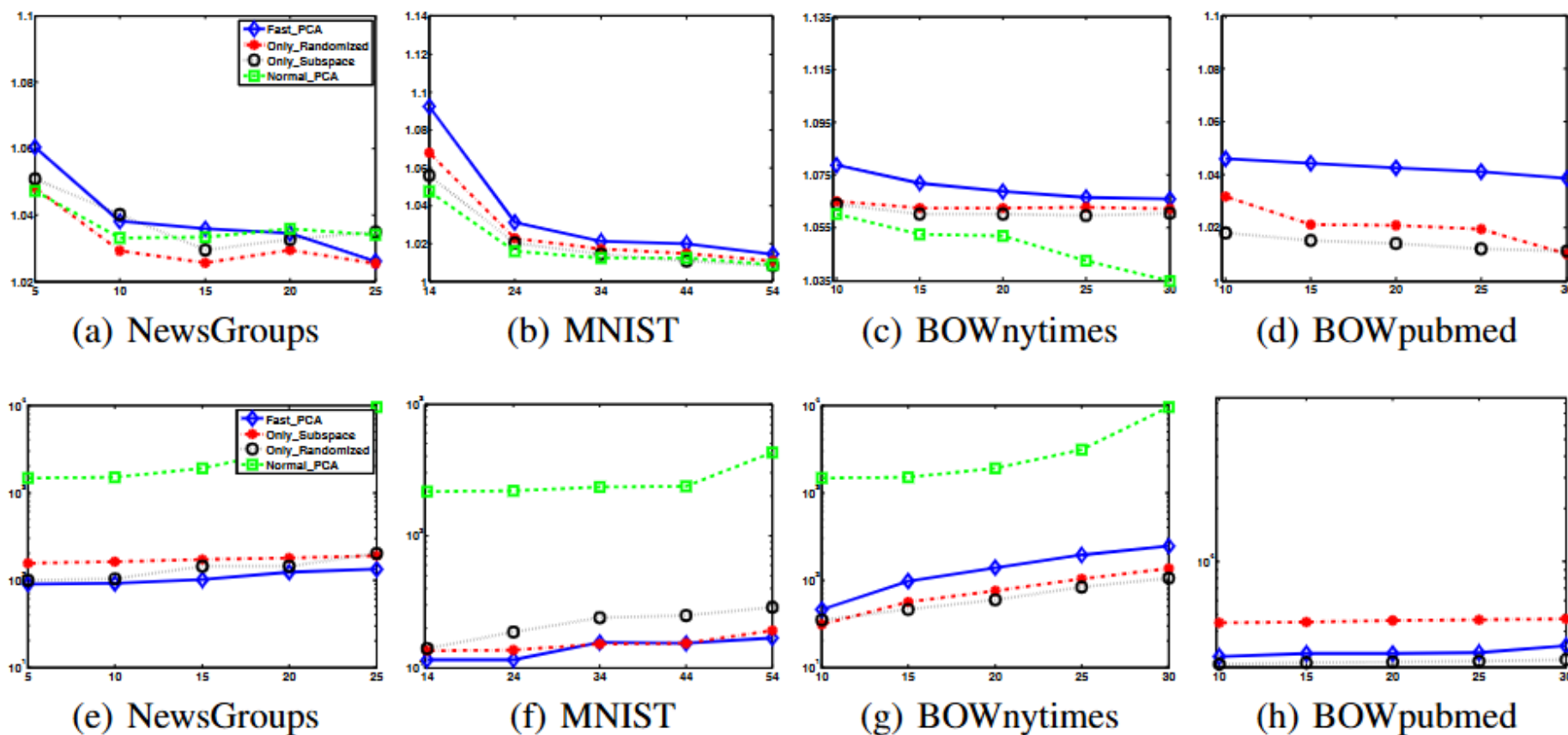


Figure 3: k -means clustering. First row: cost (normalized by baseline) v.s. projection dimension. Second row: time v.s. projection dimension.

[2]Liang Y, Balcan M F F, Kanchanapally V, et al. Improved distributed principal component analysis[J]. Advances in neural information processing systems, 2014, 27.

分布式PCA降维

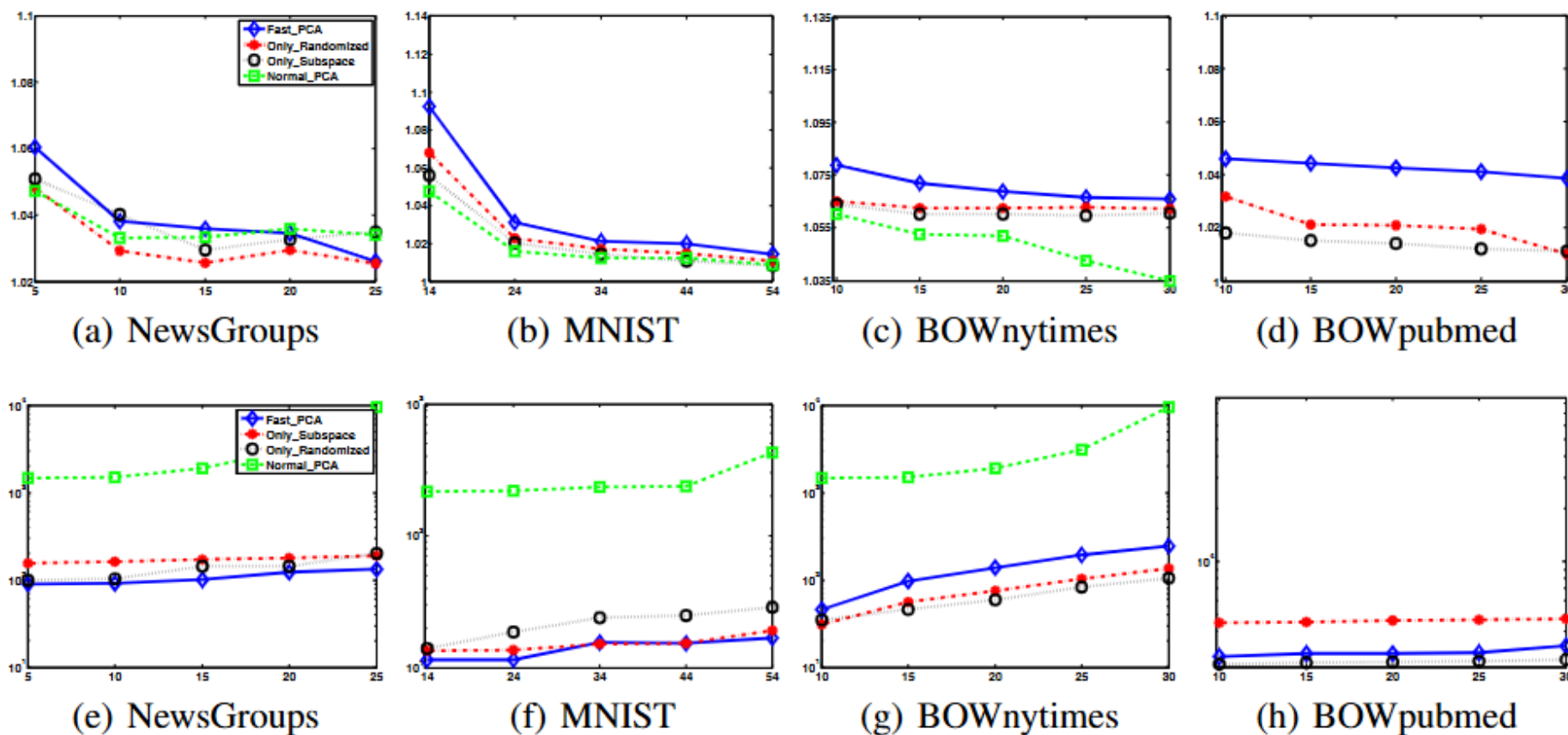


Figure 3: k -means clustering. First row: cost (normalized by baseline) v.s. projection dimension. Second row: time v.s. projection dimension.

[2]Liang Y, Balcan M F F, Kanchanapally V, et al. Improved distributed principal component analysis[J]. Advances in neural information processing systems, 2014, 27.

分布式PageRank

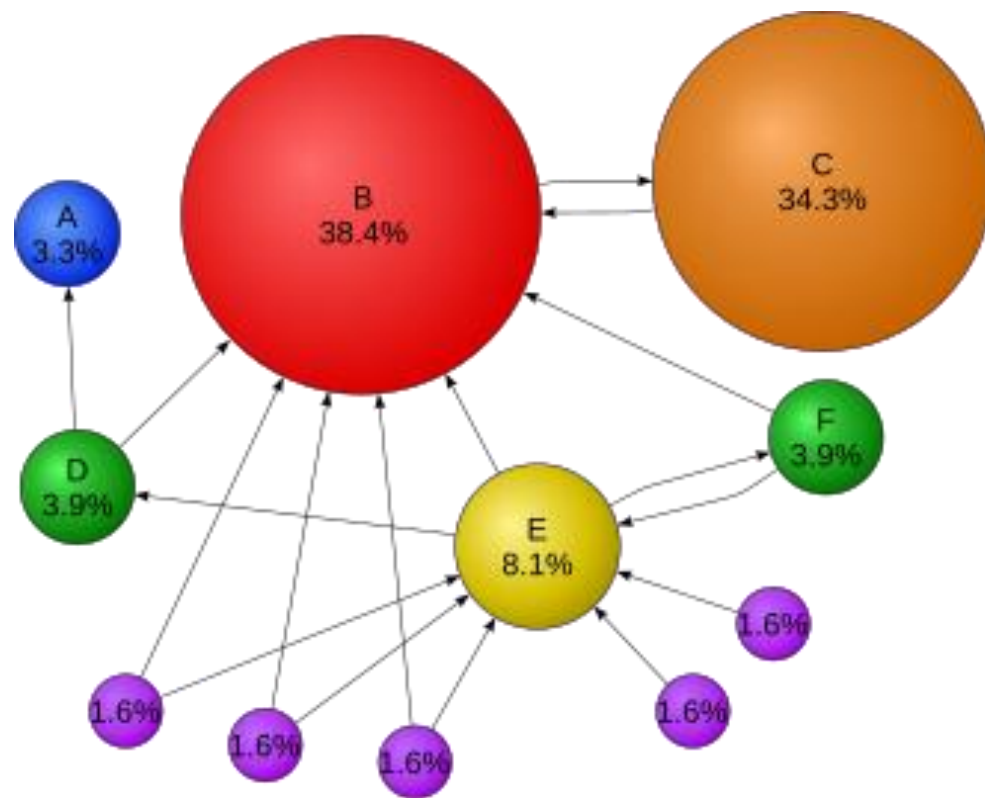
“一个网页的重要性由指向它的其他网页的重要性共同决定。”

对于有向图 $G(V, E)$ 中的一个节点 u , 它的PageRank值为:

$$PR(u) = \frac{1-d}{N} + d \cdot \sum_{v \in In(u)} \frac{PR(v)}{deg(v)}$$

其中:

- d : 阻尼系数, 常取0.85
- N : 总节点数
- $In(u)$: 指向 u 的所有页面
- $deg(v)$: 节点 v 的出度



分布式PageRank

单机版本

迭代过程中, u 在第 $t + 1$ 轮更新后的PageRank值为:

$$PR^{t+1}(u) = \frac{1-d}{N} + d \cdot \sum_{v \in In(u)} \frac{PR^t(v)}{deg(v)}$$

单机版本中, 对于每个节点 u , 需访问它的所有入边邻居 $v \in In(u)$, 总复杂度为:

$$O(|E|)$$

因为 $G(V, E)$ 中的每条边都被访问一次, 用于计算 $\frac{PR^t(v)}{deg(v)}$

分布式PageRank

分布式版本

分布式迭代过程中，机器 M_i 负责子图 $G_i(V_i, E_i)$ ，每台机器按以下步骤独立计算：

1. 本地PageRank分发： 遍历自己负责的节点 $u \in V_i$ ，将 u 的贡献 $\frac{PR^t(u)}{deg(u)}$ 分发给 u 的所有邻居
2. 跨机器通信： 若邻居 $v \in V_j \neq V_i$ ，就将贡献发送给其他机器 M_j
3. 聚合更新： 对每个本地节点 u ，汇总所有来自入邻居的贡献，更新 u 的PageRank值：

$$PR^{t+1}(u) = \frac{1-d}{N} + d \cdot \sum_{v \in In(u)} \frac{PR^t(v)}{deg(v)}$$

本地计算复杂度 $\approx$ 处理边集 E_i ；若图近似均匀划分，则每台机器每轮的**本地复杂度**：

$$O(|E|/m)$$

每轮所有机器产生的**总通信复杂度**：

$$O(\# \text{跨机器边})$$

理想划分：边不跨机器

最坏划分：每轮通信复杂度 $O(|E|)$

大数据算法：分布式算法

- 基础概念
- 算法的分布式实现
- **分布式机器学习训练**
- 联邦学习与区块链



分布式训练

生成型预训练变换模型 3 (Generative Pre-trained Transformer 3, GPT-3)

GPT-3模型规模:

- 1750亿个参数 (175 billion parameters)
- 使用了 45TB 的文本数据 作为训练集 (去重后的文本大约为 570GB)
- 训练使用了 超级计算集群, 包含了 数千个GPU, 训练时间以周为单位计算



训练所需计算量远超单机计算能力

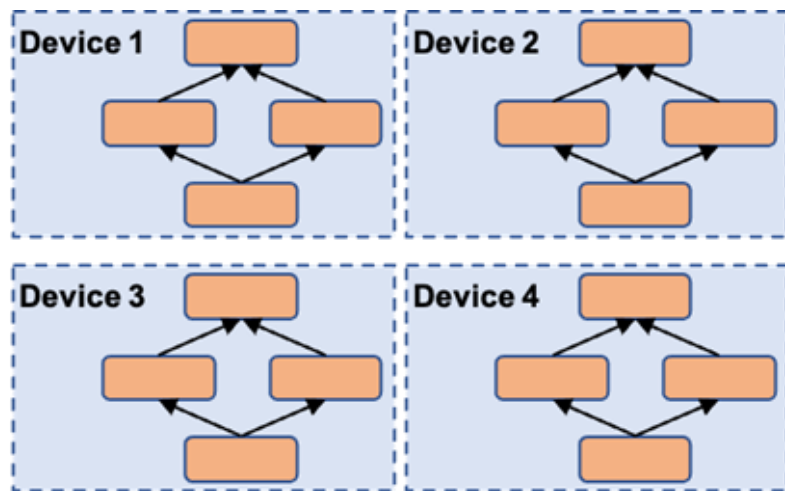


分布式训练方式

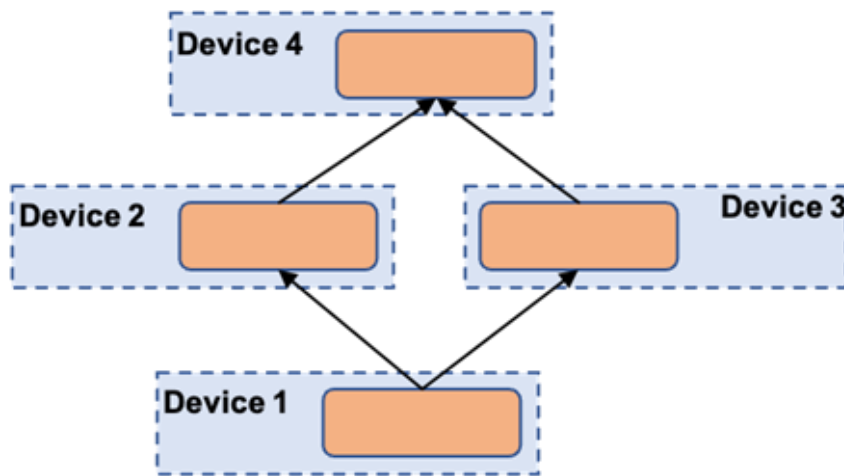
分布式训练：基本模式

数据并行 (Data Parallelism)：模型复制，数据划分。

模型并行 (Model Parallelism)：模型划分，多节点协同训练。



数据并行

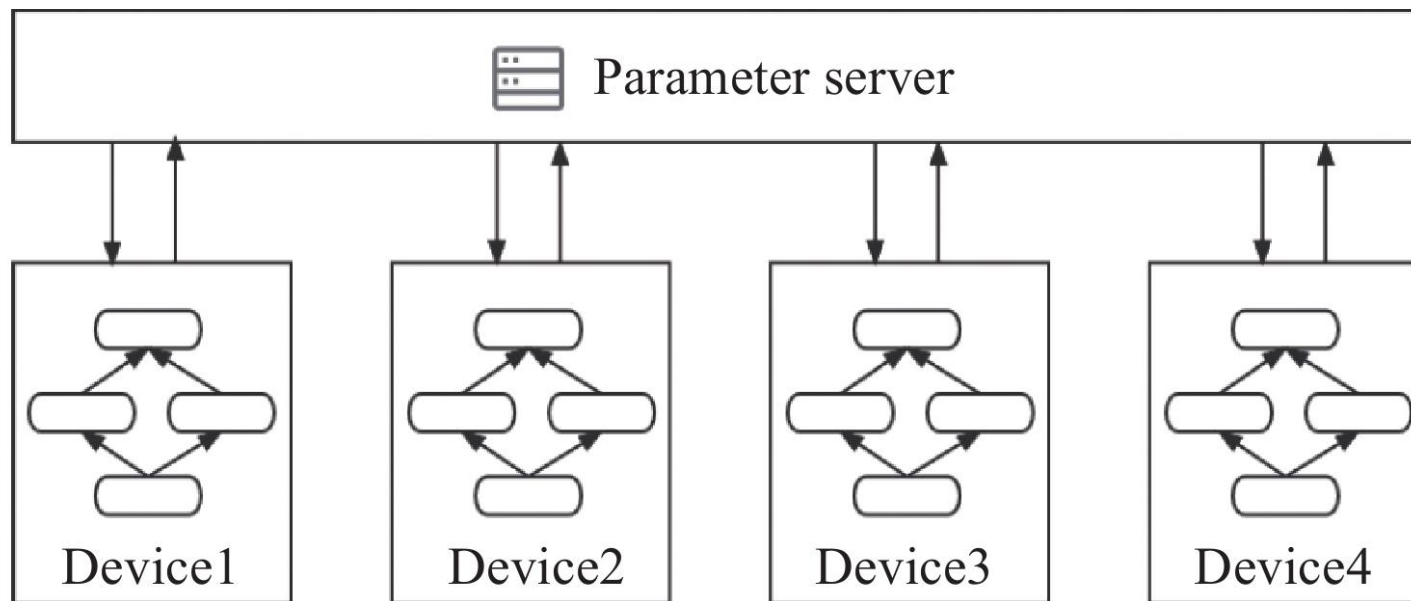


模型并行

分布式训练：系统架构

参数服务器 (Parameter Server, PS) 架构

将训练模型的参数集中管理，由一组称为**参数服务器 (PS)** 的节点负责模型参数的存储与更新，而其他节点称为**工作节点 (worker)**，它们负责数据读取与梯度计算。

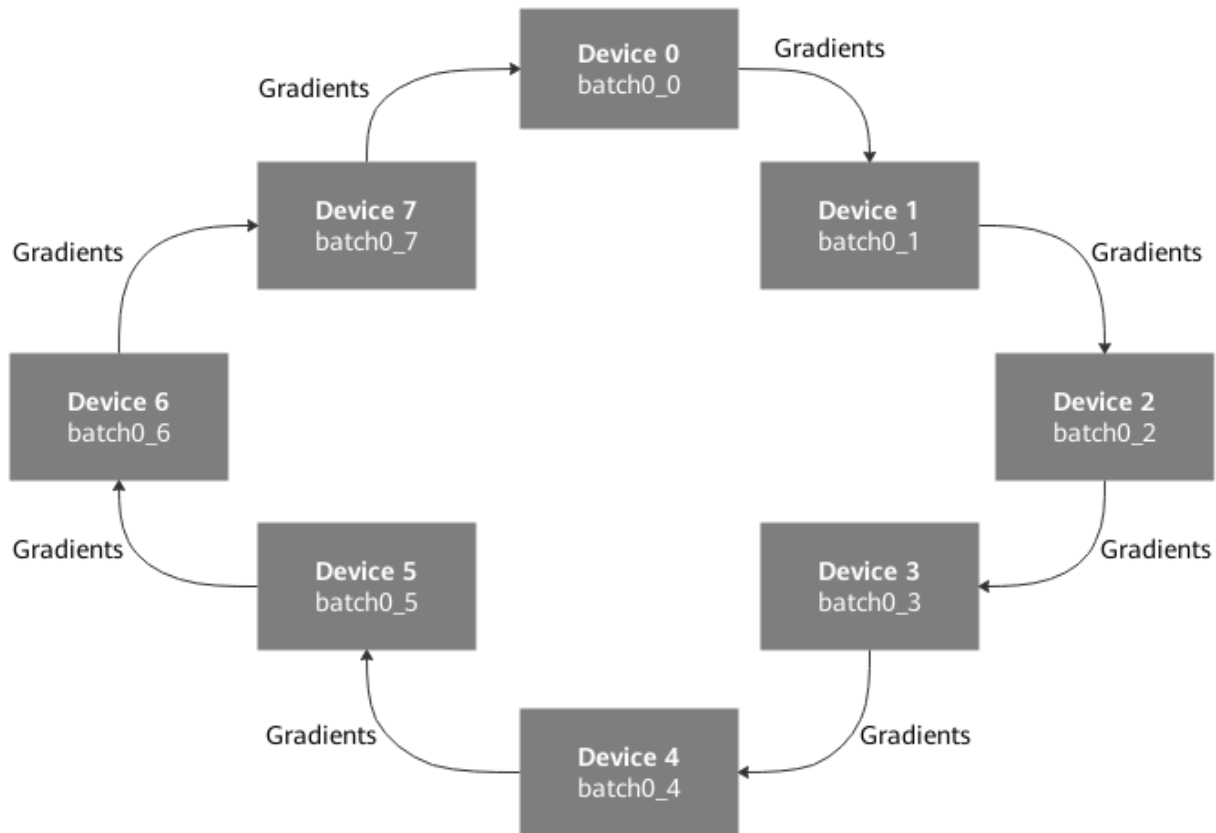


分布式训练：系统架构

AllReduce 架构 (如 Ring-AllReduce)

每个节点拥有模型的**完整副本**，训练过程通过 **AllReduce 操作** 实现各个节点间梯度的**同步聚合**，不需要中心参数服务器。

1. 每个 Worker 使用本地数据计算梯度
2. 所有节点之间通过 AllReduce 操作 (比如 NCCL库) **直接通信并聚合梯度**
3. 每个节点本地同步更新自己的模型副本



分布式训练：同步 v.s. 异步训练

同步训练 (Synchronous Training)

以PS架构为例，每轮训练中，所有 Worker 都必须完成本轮梯度计算，并将结果发送给聚合节点，**等所有人到齐后再更新模型参数**。

异步训练 (Asynchronous Training)

各个 Worker 不需要等待其他节点，谁算完了就立刻把梯度发给 PS，由 PS **立即更新模型参数**并发送给该Worker，其他节点继续按自己的节奏进行。

	同步训练	异步训练
收敛速度	稳定但较慢	较快但可能不稳定
收敛精度	高，更新一致性好	低，更新不一致
应用案例	Horovod, DDP, Ring-AllReduce	TensorFlow Async, DistBelief

分布式训练：实际例子

Async-SGD [3]: 异步训练

Algorithm 1: Async-SGD worker k

```
Input: Dataset  $\mathcal{X}$ 
Input:  $B$  mini-batch size
1 while True do
2   Read  $\hat{\theta}_k = (\theta[0], \dots, \theta[M])$  from PSs.
3    $G_k^{(t)} := 0$ .
4   for  $i = 1, \dots, B$  do
5     Sample datapoint  $\tilde{x}_i$  from  $\mathcal{X}$ .
6      $G_k^{(t)} \leftarrow G_k^{(t)} + \frac{1}{B} \nabla F(\tilde{x}_i; \hat{\theta}_k)$ .
7   end
8   Send  $G_k^{(t)}$  to parameter servers.
9 end
```

Algorithm 2: Async-SGD Parameter Server j

```
Input:  $\gamma_0, \gamma_1, \dots$  learning rates.
Input:  $\alpha$  decay rate.
Input:  $\theta^{(0)}$  model initialization.
1 for  $t = 0, 1, \dots$  do
2   Wait for gradient  $G$  from any worker.
3    $\theta^{(t+1)}[j] \leftarrow \theta^{(t)}[j] - \gamma_t G[j]$ .
4    $\bar{\theta}^{(t)}[j] = \alpha \bar{\theta}^{(t-1)}[j] + (1 - \alpha) \theta^{(t)}[j]$ .
5 end
```

对于Server:

1. 任意Worker**请求模型参数**时，拷贝当前模型参数给Worker
2. 任意Worker**上传梯度**时，立刻用设定的学习率，按照Worker的指示进行更新

对于每个Worker:

1. **“拿模型”**：向参数服务器要一份当前模型参数的完整拷贝
2. **“算梯度”**：用这份模型把手里的小批量数据（mini-batch）走一遍“前向训练+反向传播”，算出梯度
3. **“梯度上传”**：把刚算好的梯度上传到参数服务器，指导参数服务器修改参数
4. **立刻进入下一轮**：不等服务器改完模型，也不关心别的 Worker 的进度，直接去拉取新数据、拉取当前最新模型开始下一轮

[3] Chen J, Pan X, Monga R, et al. Revisiting distributed synchronous SGD[J]. arXiv preprint arXiv:1604.00981, 2016.

分布式训练：实际例子

梯度陈旧 (Staleness)

- 在异步训练中，每个 worker 先“拿模型” → “算梯度” → “梯度上传”。这段时间里，别的 worker 可能更新模型
- 当服务器收到这份梯度时，它对应的是**几步之前的旧模型**，这份梯度就被称为“陈旧”梯度，导致误差变高、收敛性差
- 可以通过**限制最大陈旧度**的Stale Synchronous Parallel (SSP)方法，最多只允许落后一定步数的梯度被应用

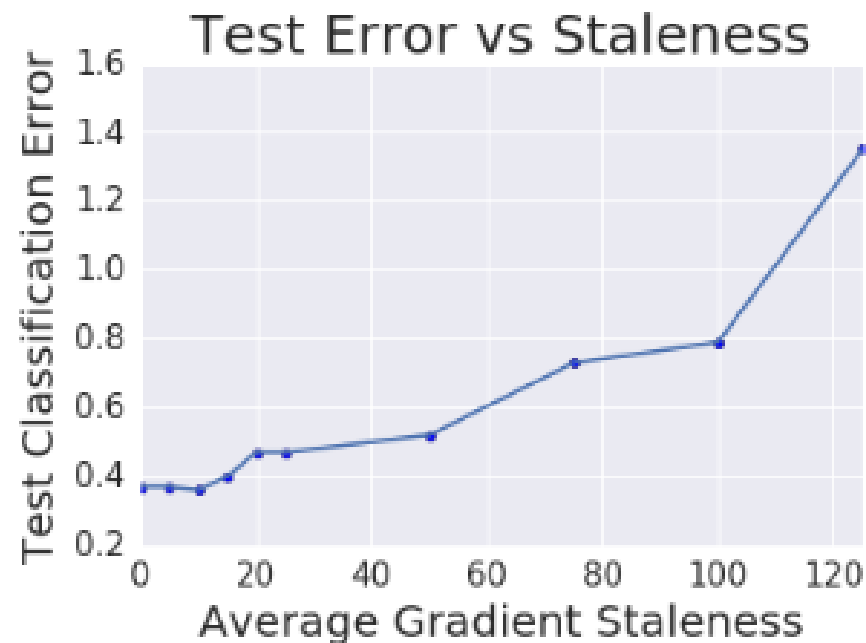


Figure 2: Degradation of test classification error with increasing average gradient staleness in MNIST CNN model.

分布式训练：实际例子

Sync-SGD [3]：同步训练

Algorithm 3: Sync-SGD worker k , where $k = 1, \dots, N + b$

Input: Dataset $\mathcal{X}$
Input: B mini-batch size

```
1 for  $t = 0, 1, \dots$  do
2   Wait to read  $\theta^{(t)} = (\theta^{(t)}[0], \dots, \theta^{(t)}[M])$ 
   from parameter servers.
3    $G_k^{(t)} := 0$ 
4   for  $i = 1, \dots, B$  do
5     Sample datapoint  $\tilde{x}_{k,i}$  from  $\mathcal{X}$ .
6      $G_k^{(t)} \leftarrow G_k^{(t)} + \frac{1}{B} \nabla F(\tilde{x}_{k,i}; \theta^{(t)})$ .
7   end
8   Send  $(G_k^{(t)}, t)$  to parameter servers.
9 end
```

Algorithm 4: Sync-SGD Parameter Server j

Input: $\gamma_0, \gamma_1, \dots$ learning rates.
Input: α decay rate.
Input: N number of mini-batches to aggregate.
Input: $\theta^{(0)}$ model initialization.

```
1 for  $t = 0, 1, \dots$  do
2    $\mathcal{G} = \{\}$ 
3   while  $|\mathcal{G}| < N$  do
4     Wait for  $(G, t')$  from any worker.
5     if  $t' == t$  then  $\mathcal{G} \leftarrow \mathcal{G} \cup \{G\}$ .
6     else Drop gradient  $G$ .
7   end
8    $\theta^{(t+1)}[j] \leftarrow \theta^{(t)}[j] - \frac{\gamma_t}{N} \sum_{G \in \mathcal{G}} G[j]$ .
9    $\bar{\theta}^{(t)}[j] = \alpha \bar{\theta}^{(t-1)}[j] + (1 - \alpha) \theta^{(t)}[j]$ .
10 end
```

对于Server:

1. “收集梯度”：进入第 t 轮后，服务器接收时间戳为 t 的上传梯度。直到接收够 N 份后，停止接收
2. 计算并发布新模型：利用 N 份梯度数据更新一次模型，并发布最近模型给所有Workers

对于每个Worker:

1. “等待模型”：等待参数服务器发布新的模型参数，然后拉取一份新模型参数的完整拷贝
2. “算梯度”：用这份模型把手里的小批量数据（mini-batch）走一遍“前向训练+反向传播”，算出梯度
3. “带时间戳的梯度上传”：把刚算好的梯度标注好轮次 t ，上传到参数服务器
4. 等待下一轮：阻塞，直到参数服务器发布新模型参数

[3] Chen J, Pan X, Monga R, et al. Revisiting distributed synchronous SGD[J]. arXiv preprint arXiv:1604.00981, 2016.

分布式训练：实际例子

慢节点 (Stragglers)

- 在同步训练中，如图所示，前 90 个梯度几乎都在 2 秒内到齐。但等到第 100 个（最后一个）梯度，尾部拖得很长——存在极少数迭代要几十甚至上百秒，说明**极个别慢节点把整轮训练卡住**。这种“长尾”现象就是 straggler 问题的根源。
- 实际部署会留出 **少量备份 worker**（比如 4 %–5 %），只聚合最快的 N 份梯度，就能在**总训练时间**上取得最优折中。

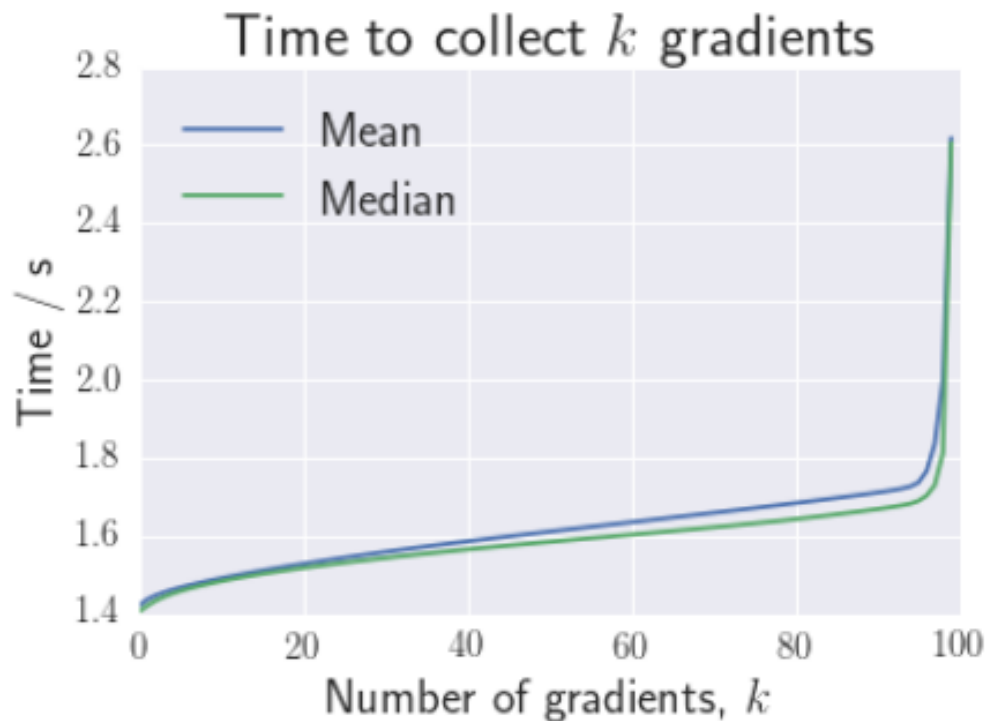
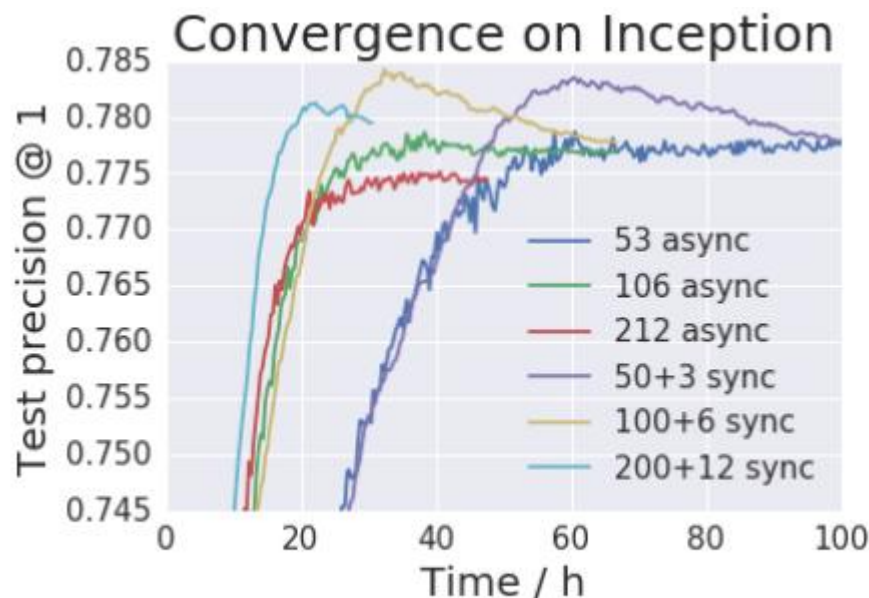
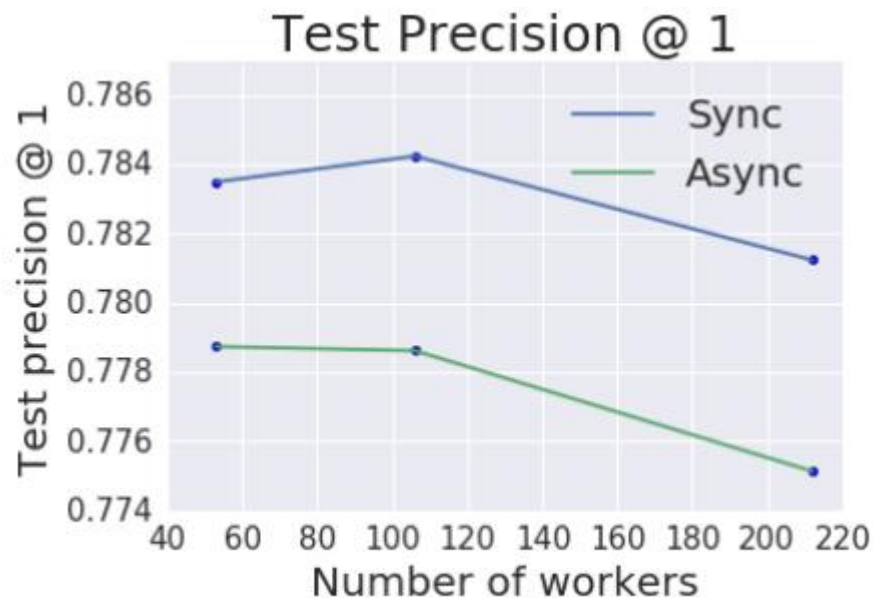


Figure 4: Mean and median times, across all iterations, to collect k gradients on $N = 100$ workers and $b = 0$ backups. Most mean times fall between 1.4s and 1.8s, except of final few gradients.

分布式训练：同步SGD v.s. 异步SGD



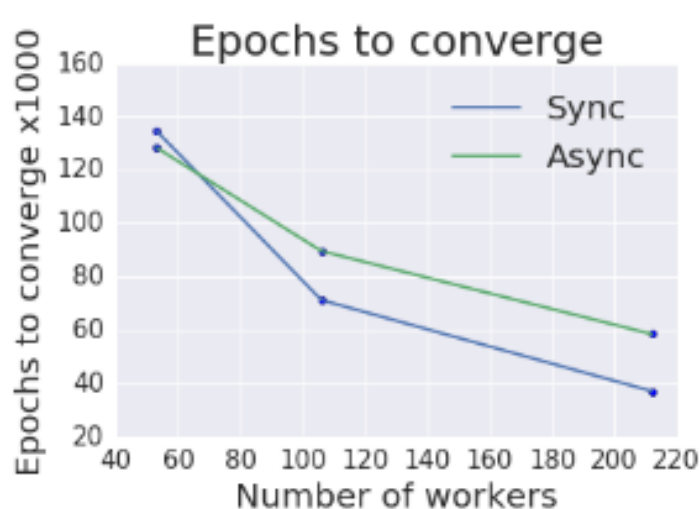
(a) Convergence



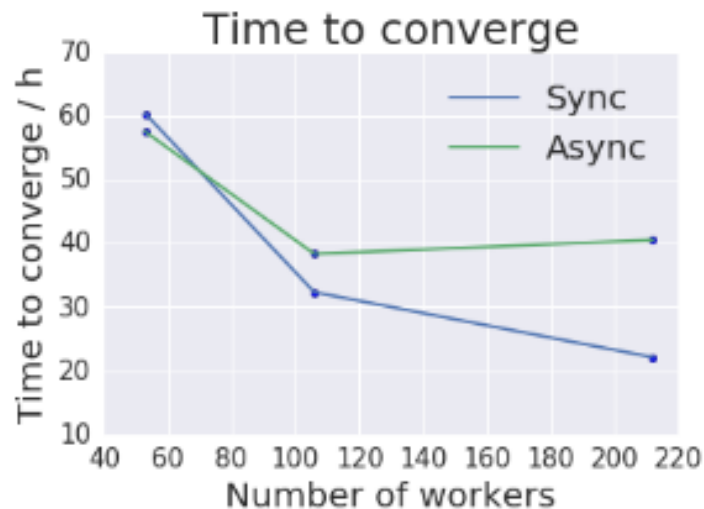
(b) Test precision @ 1

- 最终精度：同步SGD始终略胜异步SGD
- 同步训练的精度随着训练时间的增加而逐渐下降，可能存在过拟合。而异步训练自带梯度噪声，相当于一种正则化，所以曲线更平。

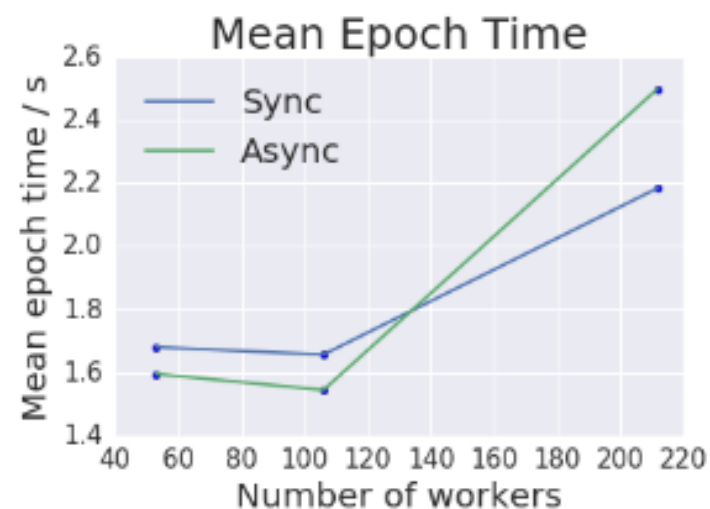
分布式训练：同步SGD v.s. 异步SGD



(c) Epochs to converge



(d) Time to converge



(e) Mean epoch time

- 收敛速度：workers个数较少时，异步训练收敛快：同步等待开销占比大
workers个数较多时，同步训练收敛快：每个epoch训练效果更好，可以用更少的epoch完成收敛

大数据算法：分布式算法

- 基础概念
- 算法的分布式实现
- 分布式机器学习训练
- 联邦学习与区块链



联邦学习

- General Data Protection Regulation (GDPR, 通用数据保护条例) 是由**欧盟**在2016年颁布, 并于**2018年5月25日正式生效**的一部里程碑式的数据隐私法规。GDPR具有两条重要核心理念: 数据不应离开数据产生地 (“数据本地化” 原则), 以及个人数据应在处理前获得授权, 且有权请求删除。
- GDPR的适用范围很广, 不仅适用于欧盟公司, 也适用于任何处理欧盟公民数据的公司, 无论公司所在地在哪。比如: 中国公司处理欧盟用户数据, 也受GDPR约束。

动机: 数据越来越分散地存储在多个设备/组织中, 例如: 智能手机、医院、银行; 且法规限制数据集中处理 (如 **GDPR**、**HIPAA**)。同时, 传统的分布式训练面临如隐私、带宽、安全成本高等问题, 这些问题导致数据 “不能动”。需要在**不集中管理数据, 且保障用户隐私**的前提下完成机器学习任务。

联邦学习 v.s. 传统分布式训练

PS架构

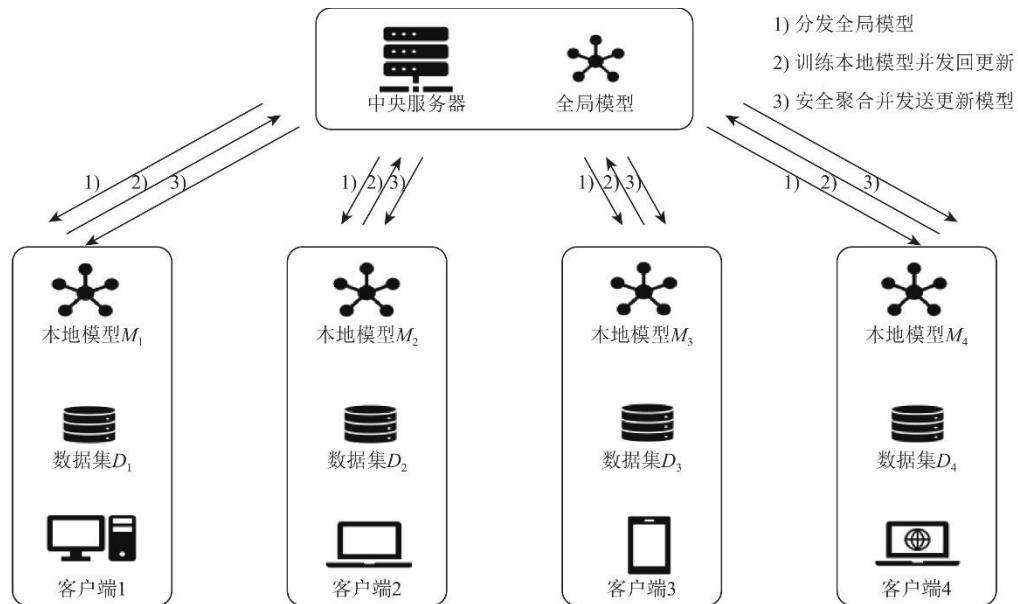
1. 将整个训练数据划分给不同 Worker
2. 训练循环：每个 Worker 拉取最新模型
3. 用本地数据计算梯度
4. 将梯度上传给 PS
5. PS 聚合所有梯度并更新模型
6. 将更新后的模型广播给所有 Workers

FedSGD架构

- **差分隐私 (Differential Privacy)**: 客户端在上传梯度前剪裁梯度并加入噪声，使得攻击者无法根据上传信息反推出原始数据。理论上，差分隐私可以量化隐私泄露概率 (ϵ -differential privacy)
- **安全多方计算 (Secure Multi-Party Computation, MPC)**: 客户端之间使用secret sharing机制进行梯度共享使得服务器无法看到任意单个梯度，只能看到聚合结果。
- **同态加密 (Homomorphic Encryption)**: 客户端用加密算法对梯度加密，服务器直接在密文上进行加法聚合，不解密也能完成更新。

联邦学习

FedSGD 的隐私保护不是自动的，真正的安全特性依赖于：差分隐私、加密、MPC 等联邦学习生态中的扩展机制。

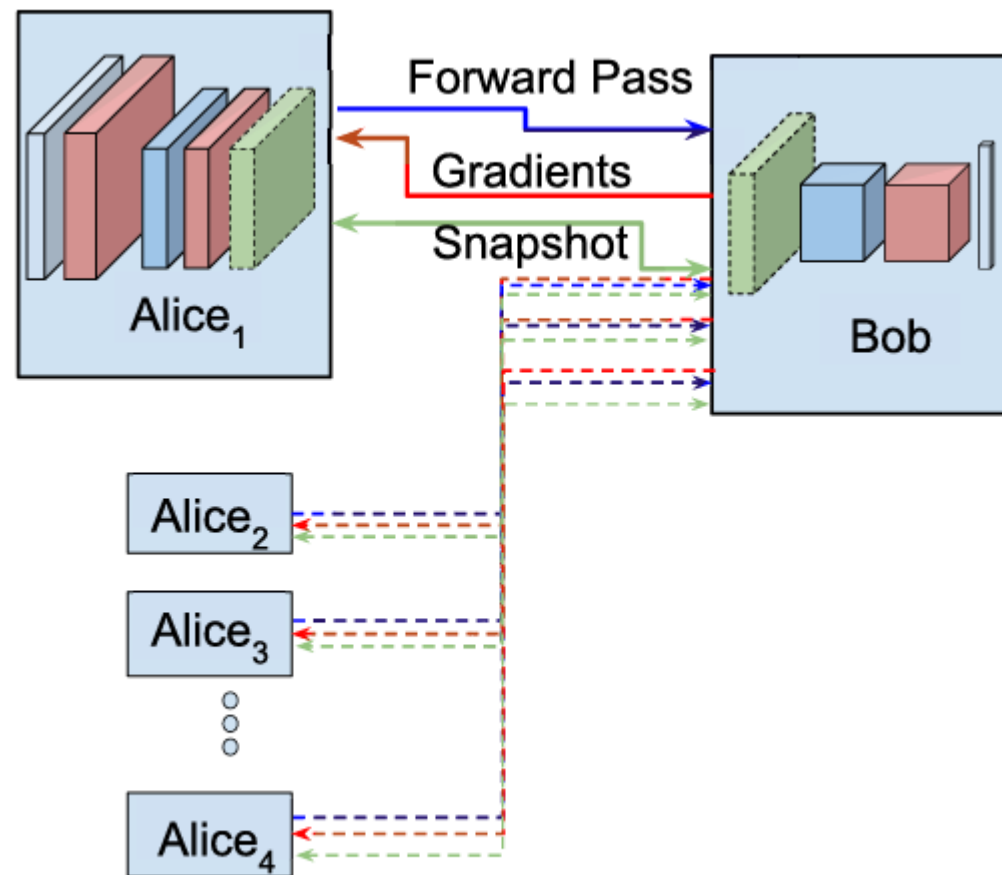


	FedSGD	参数服务器 (PS)
网络环境	异构、跨组织、广域网	同质、数据中心内部局域网
节点信任关系	客户端不可信	节点间互相信任
数据归属	数据永远在客户端	数据由系统划分后分配到 Worker
通信模式	低频、容错、延迟容忍	高频、实时
设计动机	隐私保护、协作学习	并行训练、性能提升

联邦学习：实际例子—Split Neural Network [4]

Split NN的训练过程—中心化联邦学习

- 串行训练：当前正在参与训练的那个Alice持有神经网络的前 n 层 $L_0, \dots, L_n$ 的最新参数。Bob始终持有神经网络的所有层的最新参数，包括前 n 层 $L_0, \dots, L_n$ 和后 $N - n$ 层 $L_{n+1}, \dots, L_N$
- 1. **“拿模型”**：每个 $Alice_j$ 在开始一次训练前，先从Bob处拉取网络前 n 层 $L_0, \dots, L_n$ 的最新参数 (Snapshot)。
- 2. **“合作前向传播”**： $Alice_j$ 使用自己的本地数据，对网络的前 $L_0, \dots, L_n$ 层执行前向传播，得到第 n 层的输出，发给Bob。Bob继续前向传播得到标签



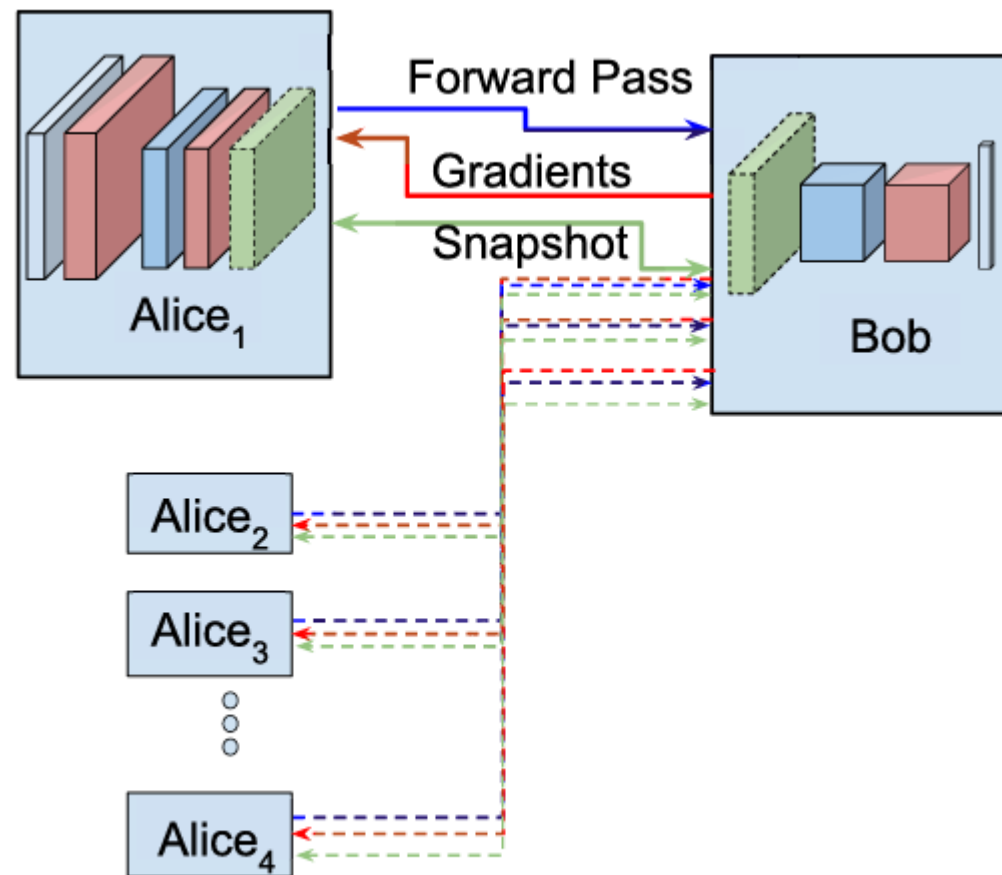
(a) Centralized distributed neural network training.

[4] Gupta O, Raskar R. Distributed learning of deep neural network over multiple agents[J]. Journal of Network and Computer Applications, 2018, 116: 1-8.

联邦学习：实际例子—Split Neural Network [4]

Split NN的训练过程—中心化联邦学习

3. **“合作反向传播”**：Bob根据标签计算损失并执行反向传播，直到第 $n+1$ 层 L_{n+1} ，产生对第 n 层输出的梯度。 $Alice_j$ 接收到对第 n 层输出的梯度后，从 $L_n \rightarrow L_0$ 执行自己的反向传播并更新参数。
 4. **“上传模型”**： $Alice_j$ 把网络前 n 层 $L_0, \dots, L_n$ 的最新参数上传给Bob，确保Bob始终持有神经网络的所有层的最新参数。
- 同一时间最多只能有一个 $Alice_j$ 进行训练，牺牲了并行性，换来了隐私保护、结构分离和设备适应性。



(a) Centralized distributed neural network training.

[4] Gupta O, Raskar R. Distributed learning of deep neural network over multiple agents[J]. Journal of Network and Computer Applications, 2018, 116: 1-8.

联邦学习：实际例子—Split Neural Network [4]

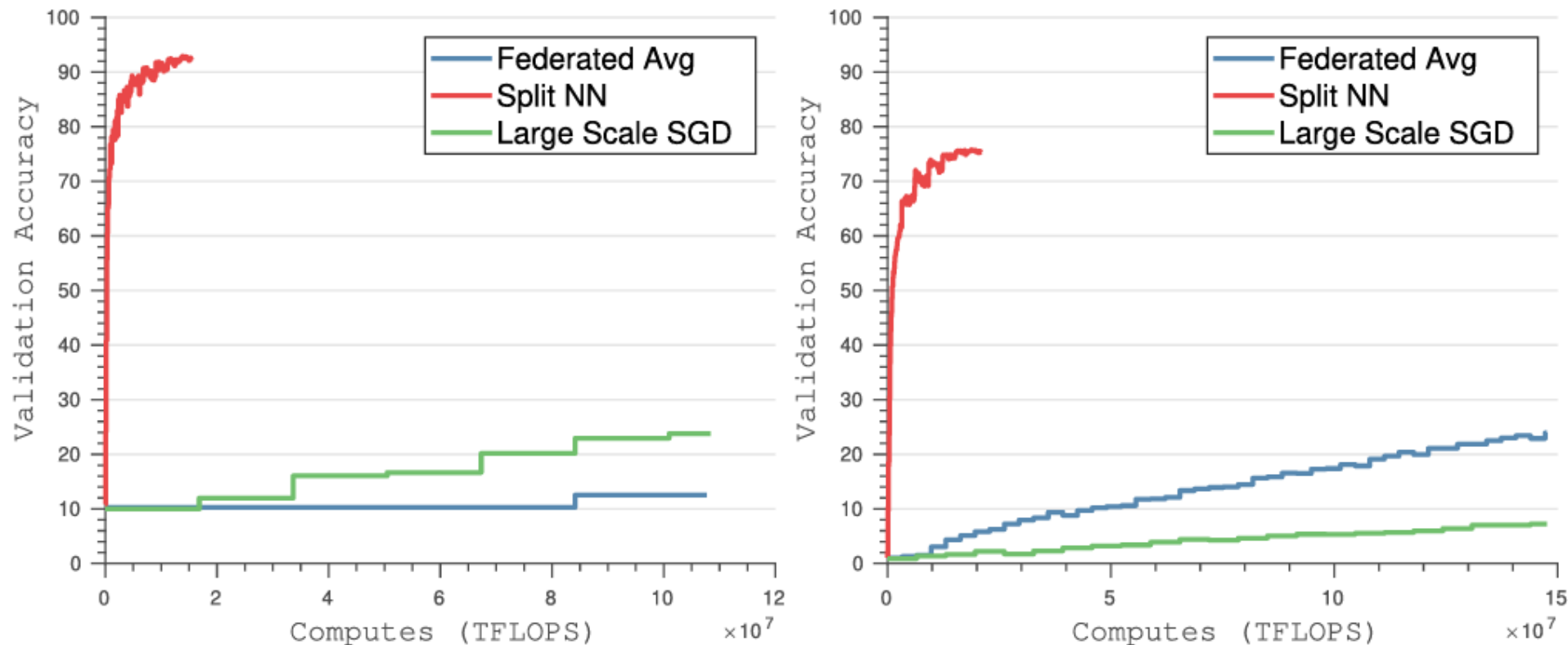
Split NN的特点：隐私保护

- **对比传统分布式神经网络：**传统分布式神经网络Worker上传完整梯度，理论上可被反演出敏感信息；Split NN只暴露更“抽象”的单层特征和单层梯度，因此隐私泄露面更小。
- **可选保护手段：**在特征或梯度上加噪声（差分隐私）、量化、加密。进一步降低重建风险。

Split NN的特点：设备适应性

- **低算力：**对于低计算能力的客户端，客户端可以只计算前几层的前向/反向传播，占用CPU/GPU较少
- **低存储能力：**客户端可以只存储前几层的参数，其余参数全在服务器
- **异构设备：**不同设备的客户端可选不同 n 值（动态切分网络），算法逻辑保持不变

联邦学习：实际例子—Split Neural Network [4]



(a) Validation accuracy with client side flops when training 100 clients (VGG and CIFAR 10).

(b) Validation accuracy with client side flops when training 500 clients (Resnet-50 and CIFAR 100).

Fig. 3. Comparison of client side computational cost of our method against existing state of the art methods.

[4] Gupta O, Raskar R. Distributed learning of deep neural network over multiple agents[J]. Journal of Network and Computer Applications, 2018, 116: 1-8.

区块链

一种去中心化的分布式账本技术。数据由多个节点共同维护，无需依赖中心服务器。每条记录打包进区块，并按时间顺序连接形成链。一旦数据上链，即不可更改，确保安全与可追溯。

视频简介



- 每个区块包含：交易数据 + 上一个区块的哈希 + 时间戳。
- 节点使用共识机制（如PoW工作量证明）达成共识。
- 新块通过验证后添加到链中，形成不可篡改记录。
- 每个节点维护一份完整账本，确保数据一致性。